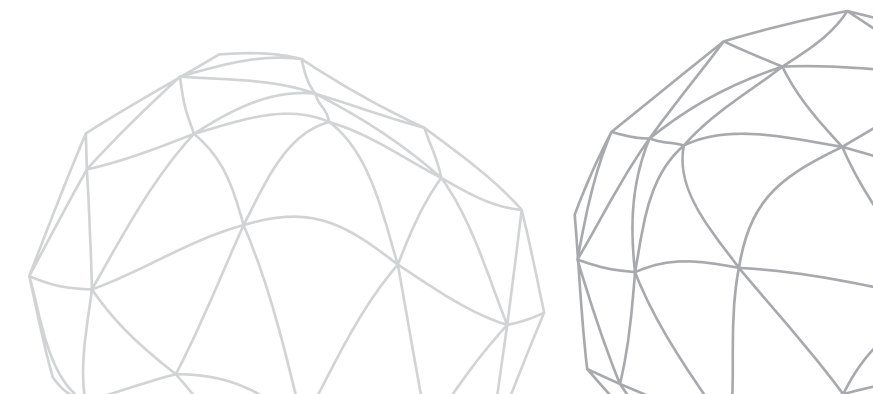


Quantum Machine Learning

Quantum Algorithms for solving the XOR problem

Daniel Paiva, Gabriel Ferreira
2026



Why Quantum Computing?

- The notion of a universal computer was first characterized by the description of the **Turing Machine**, based on a computer register, which through the machine could be used to encode any algorithm;
- The transistor led computing's architecture we have today. However, with the trend of increasing transistor density per unit area, eventually a limit could be perceived, called the **Moore Law**;
- In that way, looking for **alternative architectures** are inevitable since the polynomial time computational complexity of this computing architecture is a real limit;
- For that there is an idea thought by Richard Feynman that could be an alternative for von Neumann architecture. This idea considers using **quantum phenomena**, such as superposition and entanglement, for computing;

Why Quantum Computing?

- The goal is to construct algorithms which yield combinations of operators that use sequential constructive and destructive interference of wave function to promote the correct solution probability while cancelling out the probability of incorrect solutions;
- **Many companies and organizations are exploring** numerous computing architectures, each trying to establish their approach as the dominant one;
- Other discussions beyond hardware implementations, there are other propositions, like building quantum computing on **qudits** (d-level quantum bits) in place of qubits and building computing on a **continuum quantum state**;
- Beyond the goal of **universal quantum computing** are also specific quantum realizations for specific applications, like **quantum annealing** for optimization problems and **quantum machine learning** problems;
- Big investments in quantum computing has led to a race for a **quantum supremacy**. The expectation is that this supremacy **can be achieved early in the 2030's**;

Quantum Computing

- The **mathematics of quantum mechanics** is construct with **linear algebra** and **Dirac's notation**. The idea is to represent a state of a particle like an electron or a photon as

$$|\psi\rangle = \begin{bmatrix} a \\ b \end{bmatrix}$$

where $|\psi\rangle$ is an element of the **Hilbert Space** H and is called **ket**. The hermitian adjunct of $|\psi\rangle$ is the complex conjugate of transpose matrix above

$$\langle\psi| = [a^* \quad b^*],$$

with $\langle\psi|$ being an element of the dual space H^* and is called **bra**. In this way we'll have the **bra-ket**

$$\langle\psi_1|\psi_2\rangle = [a_1^* \quad b_1^*] \cdot \begin{bmatrix} a_2 \\ b_2 \end{bmatrix} = a_1^* a_2 + b_1^* b_2$$

Quantum Computing

- **Unitary operators** are used to change the quantum state, they are represented as matrices and for an operator to be classified as a unitary operator it is needed that

$$\hat{U}\hat{U}^\dagger = \hat{U}^\dagger\hat{U} = \hat{I},$$

- Applying the operator on the state $|\psi\rangle$ we'll have:

$$|\psi_{i+1}\rangle = \hat{U}|\psi_i\rangle = \begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \end{bmatrix} \begin{bmatrix} a_i \\ b_i \end{bmatrix} = \begin{bmatrix} u_{11}a_i + u_{12}b_i \\ u_{21}a_i + u_{22}b_i \end{bmatrix} = \begin{bmatrix} a_{i+1} \\ b_{i+1} \end{bmatrix}.$$

- The **hermitian** of $\hat{U}$ is described as:

$$\hat{U}^\dagger = \begin{bmatrix} u_{11}^* & u_{21}^* \\ u_{12}^* & u_{22}^* \end{bmatrix}$$

and $\hat{U}^\dagger$ is applied to $\langle\psi|$ like:

$$\langle\psi_{i+1}| = \langle\psi_i|\hat{U}^\dagger = [a_i^* \quad b_i^*] \begin{bmatrix} u_{11}^* & u_{21}^* \\ u_{12}^* & u_{22}^* \end{bmatrix} = [a_i^*u_{11}^* + b_i^*u_{12}^* \quad a_i^*u_{21}^* + b_i^*u_{22}^*] = [a_{i+1}^* \quad b_{i+1}^*]$$

Quantum Computing

- A **Hermitian operator** is defined as

$$\hat{A} = \hat{A}^\dagger$$

all eigenvalues of this type of operator are real, and all operators that represent measurable observables are Hermitian.

- The **average value** of a generic operator is

$$\langle \hat{A} \rangle = \langle \psi | \hat{A} | \psi \rangle$$

where $|\psi\rangle$ is the quantum state before its measurement, representing the average measurement outcome;

- The **bases of states** $|\psi\rangle$ are $|0\rangle$ and $|1\rangle$, so

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and} \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \langle 0|0\rangle = 1 \quad \text{and} \quad \langle 1|1\rangle = 1$$

$$|\psi\rangle = \begin{bmatrix} a \\ b \end{bmatrix} = a|0\rangle + b|1\rangle = a \begin{bmatrix} 1 \\ 0 \end{bmatrix} + b \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \langle 0|1\rangle = 0 \quad \text{and} \quad \langle 1|0\rangle = 0$$

Quantum Computing

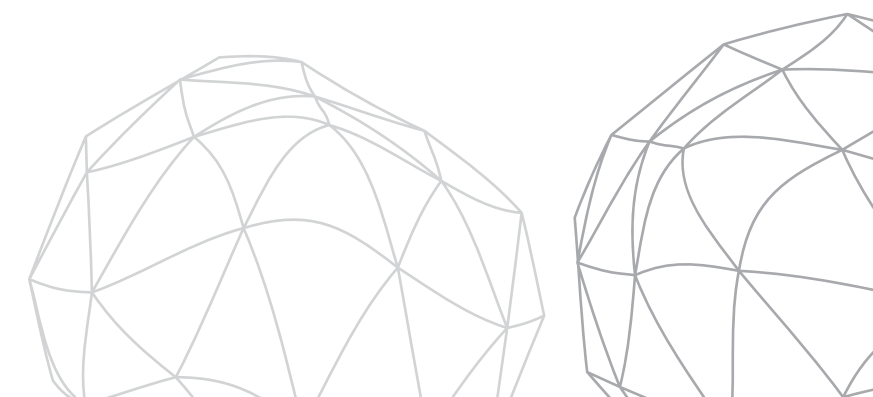
- A **Hermitian operator** is defined as



$$|\langle 0|\psi\rangle|^2 = [a^* \quad b^*] \begin{bmatrix} 1 \\ 0 \end{bmatrix} [1 \quad 0] \begin{bmatrix} a \\ b \end{bmatrix} = [a^* \quad b^*] \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = [a^* \quad b^*] \begin{bmatrix} a \\ 0 \end{bmatrix} = a^*a$$

$$|\langle 1|\psi\rangle|^2 = [a^* \quad b^*] \begin{bmatrix} 0 \\ 1 \end{bmatrix} [0 \quad 1] \begin{bmatrix} a \\ b \end{bmatrix} = [a^* \quad b^*] \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = [a^* \quad b^*] \begin{bmatrix} 0 \\ b \end{bmatrix} = b^*b$$

$$\langle \psi|\psi\rangle = [a^* \quad b^*] \begin{bmatrix} a \\ b \end{bmatrix} = a^*a + b^*b = 1$$



Quantum Computing

- **Superposition** is the state of a particle in which it exists in **two different states at the same time**;
- An example is the description of the **spin of an electron or a photon**, where superposition occurs when the spin is simultaneously **up** ($|+\rangle$) and **down** ($|-\rangle$)

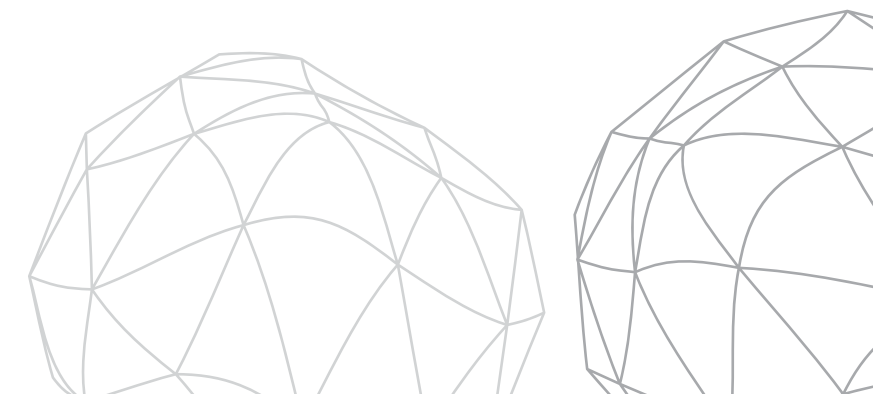
$$|s\rangle = \alpha|+\rangle + \beta|-\rangle$$

where $\alpha, \beta \in \mathbb{C}$ and

$$|\alpha|^2 + |\beta|^2 = 1$$

- At the **moment of measurement**, the wave function that represents **the superposition collapses**, and $|s\rangle$ assumes the value $|+\rangle$ or $|-\rangle$ with probabilities

$$P(|+\rangle) = |\alpha|^2; P(|-\rangle) = |\beta|^2$$



Quantum Computing

- If the system has **two particles**, their **spins may become correlated** through the phenomenon called **entanglement**;
- In this scenario, all spin directions are described as a **single quantum state** with its own properties, rather than as independent particle states. In Dirac notation, the state is written as

$$|s_1\rangle = \frac{1}{\sqrt{2}}|+\rangle + \frac{1}{\sqrt{2}}|-\rangle; |s_2\rangle = \frac{1}{\sqrt{2}}|-\rangle + \frac{1}{\sqrt{2}}|+\rangle$$

with

$$|s_1, s_2\rangle = \frac{1}{\sqrt{2}}|+, -\rangle + \frac{1}{\sqrt{2}}|-, +\rangle$$

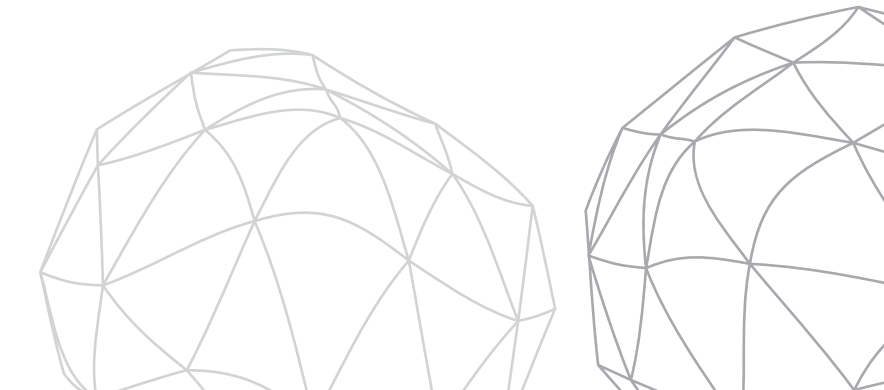
- The particles are maximally entangled when the probability of all the states outcomes are equal. An example of maximally entangled states in two dimensions are the **Bell States**;
- The Entanglement is said to be a **non-local phenomenon** because the measurement of one particle gives complete information of the other particle;

Quantum Computing



- In quantum computing, the **unit of processing is the qubit**. The difference between a classical bit and a qubit is that the phenomenon of superposition allows the qubit to **exist in more than two states during computation**;
- These states are represented by the linear combination below, where $|0\rangle$ and $|1\rangle$ are the quantum analogs of the classical bits 0 and 1.

$$|s\rangle = \alpha|0\rangle + \beta|1\rangle$$

- One fundamental condition for the quantum mechanics is that the **operations must be reversible**. This implies that every quantum logical port have to be one-to-one. Other condition is that the quantum logical ports have to **conserve the numbers of qubits**;
 - Furthermore, it must **preserve the normalization** of the quantum state, which means that all the quantum **logical ports have to be unitary operators**;
- 

Quantum Computing

- **X Gate** or **NOT Gate**: Transforms $|0\rangle$ into $|1\rangle$ and vice-versa;
- **Z Gate**: Preserves $|0\rangle$ and transform $|1\rangle$ into $-|1\rangle$;
- **H Gate**: Entangles the states, transforming $|0\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|1\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$;
- **S Gate**: Preserves $|0\rangle$ and transform $|1\rangle$ into $i|1\rangle$;
- **The $\frac{\pi}{8}$ Gate**: Preserves $|0\rangle$ and transforms $|1\rangle$ into $e^{i\frac{\pi}{4}}|1\rangle$;
- The rotation gates can be represented by matrices as

$$Rx(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}, Ry(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}, Rz(\theta) = \begin{bmatrix} e^{-i\left(\frac{\theta}{2}\right)} & 0 \\ 0 & e^{i\left(\frac{\theta}{2}\right)} \end{bmatrix}$$

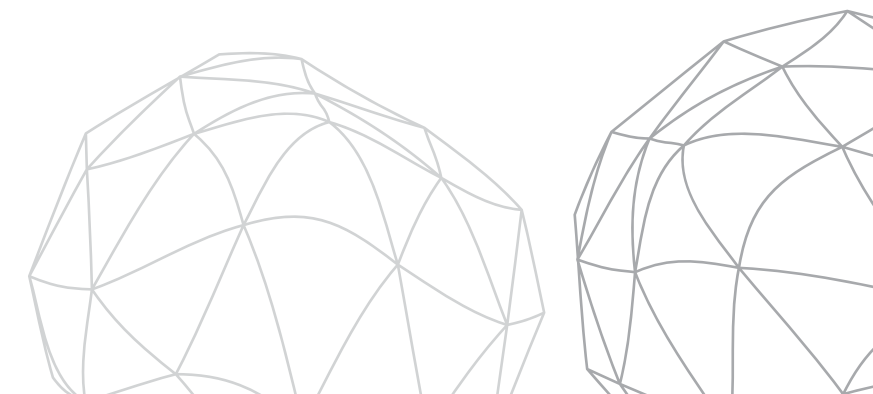
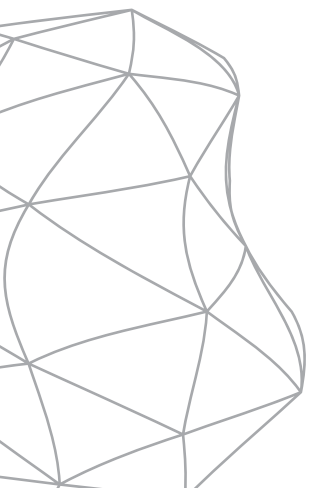
Quantum Computing

- In the **case of two qubits there are four outcomes**, given by $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$, and the **general state is**

$$|s\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle$$

- The measurement of the state is performed on the components defined by the qubits;
- For example, the probability to get $|0\rangle$ in the first qubit is $|\alpha_{00}|^2 + |\alpha_{01}|^2$ and the final state is

$$|s'\rangle = \frac{\alpha_{00}|00\rangle + \alpha_{01}|01\rangle}{\sqrt{|\alpha_{00}|^2 + |\alpha_{01}|^2}}$$



Quantum Computing

- The **two-qubit gates** are defined by 4×4 unitary matrices. Some examples include

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad \text{swap} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{controlled-phase} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & i \end{bmatrix}$$

- The **CNOT gate** changes the bit value of the other system when the value of the first is 1, as $|10\rangle \rightarrow |11\rangle$, or simply put

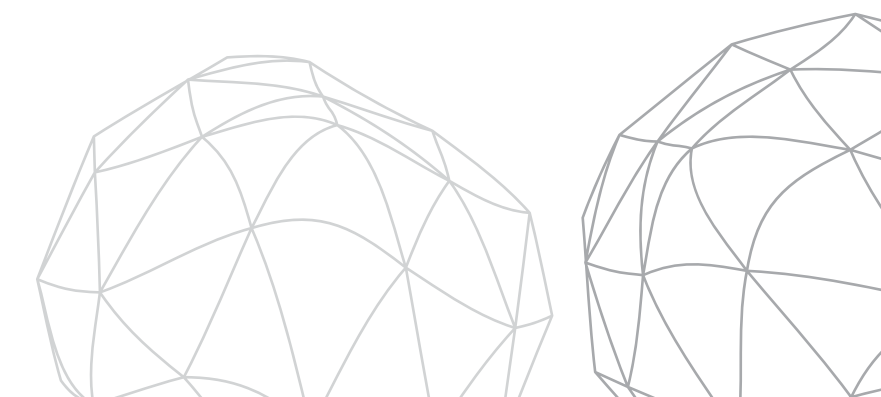
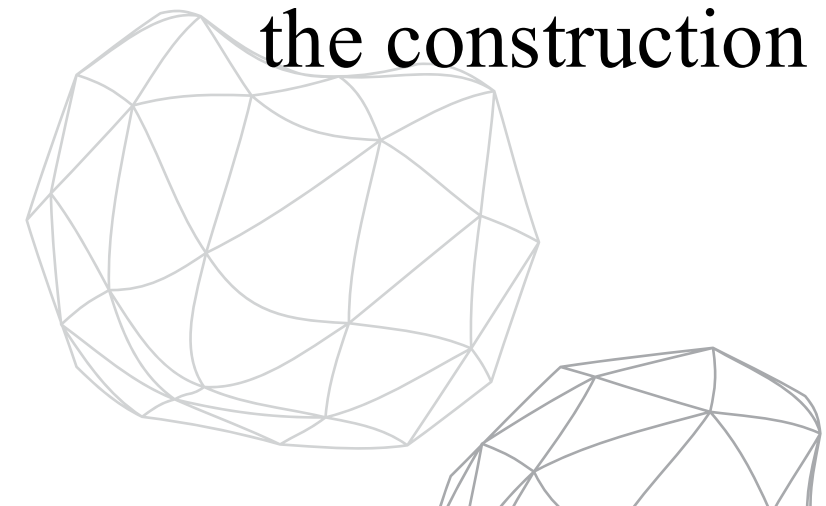
$$\text{CNOT} \begin{bmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \\ \delta \\ \gamma \end{bmatrix}, \quad \forall \alpha, \beta, \gamma, \delta \in \mathbb{C} \text{ s.t. } |\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1.$$

Quantum Computing

- There is an **infinite number of possible single-qubit gates**, so it is very convenient to have a set of gates capable of approximating any other possible gate with sufficient precision;
- This set usually includes **gates capable of creating superposition, entanglement, and or complex amplitudes**;
- An example of a set containing all these properties is $\{CNOT, S R_x\left(\frac{\pi}{4}\right)\}$ although it is **not the only possible set**;

Quantum Computing

- In classical computing, errors are corrected through redundancy. In quantum computing, however, **errors are more complex because qubits can exist in a superposition of states;**
- While a **classical bit is analogous to a π – rotation** around the X – axis on the Bloch sphere, **a qubit may undergo unwanted rotations** around any axis in three-dimensional space. This requires the **correction of phase errors and bit-flip errors;**
- In larger quantum circuits, **other errors may also arise from unwanted entanglement** between qubits;
- Nowadays, one of the main challenges is the **development of quantum error-correcting codes capable of reducing the number of physical qubits** required to create a logical qubit, as well as the construction of **hardware with sufficient tolerance** to perform useful computations;



Classical Deep Learning

- **Machine learning is use to optimally solve hard tasks**, by means of learning patterns from data;
- These tasks drew heavy inspiration from the every day tasks performed by the animal brain;
- There are **two types** of machine learning, **supervised** and **unsupervised**, where in the unsupervised only the inputs are known and the goal is to better describe the data itself, in the supervised case the inputs and outputs are known and the goal is to correctly predict the results;
- The rules are not programmed, but automatically adjusted and optimized via internal parameters that fits the data as closely as possible, minimizing the measure of distance, or error, between the predicted labels $\hat{y}$ and the true labels y of the training data x as

$$\theta = \arg \min \mathcal{F}(\hat{y}(\theta, x), y)$$

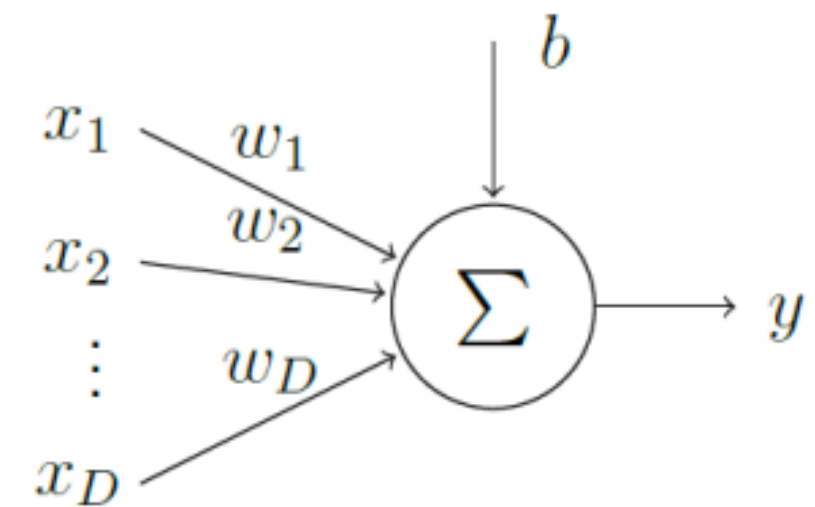
where $\mathcal{F}(\hat{y}(\theta, x), y)$ is the function to be minimized.

Classical Deep Learning

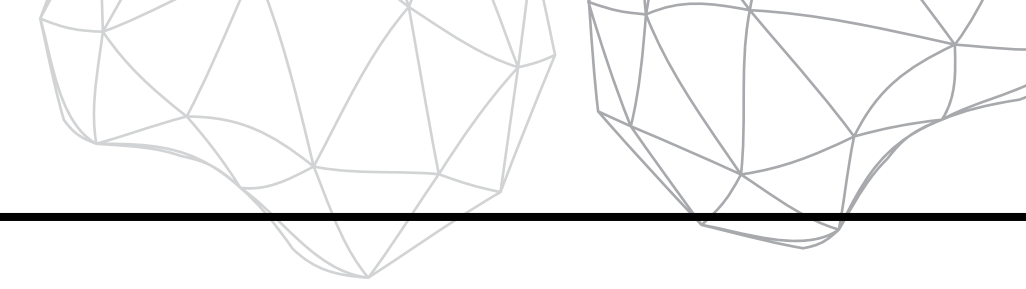
- The data is divided into **three categories**: training data, validation data and test data. During training the validation data is used for the validation loss, monitored over iterations, to avoid overfitting or underfitting;
- The final step is the testing phase, to evaluate the true performance of the model, using a completely blind set of data;
- The **perceptron is the simplest model of an artificial neural network**, that performs a linear combination of the input data, **mimicking an organic neuron**, consisting in

$$y = \sum_{i=1}^D x_i w_i + b$$

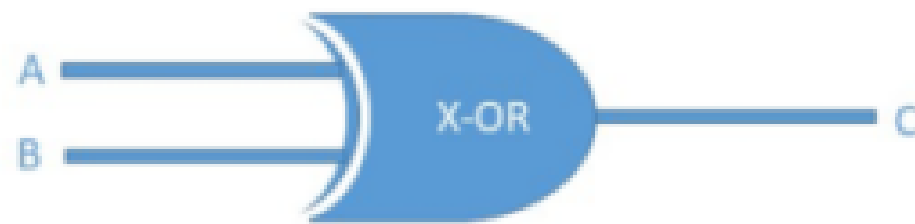
creating a linear boundary to separate different data classes.



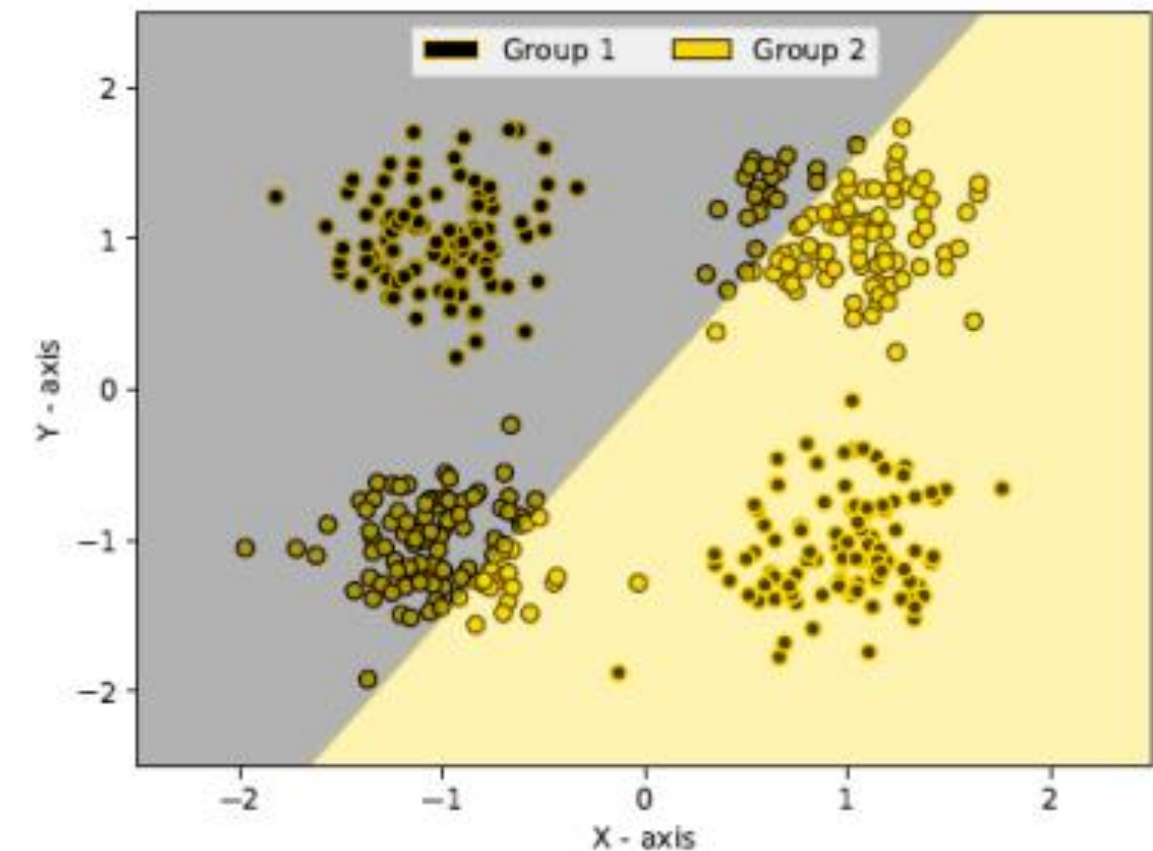
Classical Deep Learning



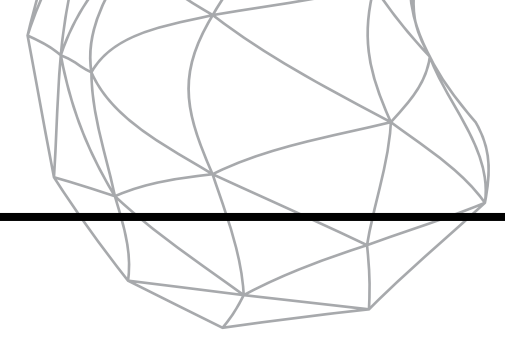
- The **XOR problem** is a classical example of the limitations of perceptrons and early neural networks, because of its non linearity;
- Its a **logical operation that consists in four clusters of data**;
- A single perceptron fails because it can only generate linear decision boundaries;
- More sophisticated algorithms are necessary.



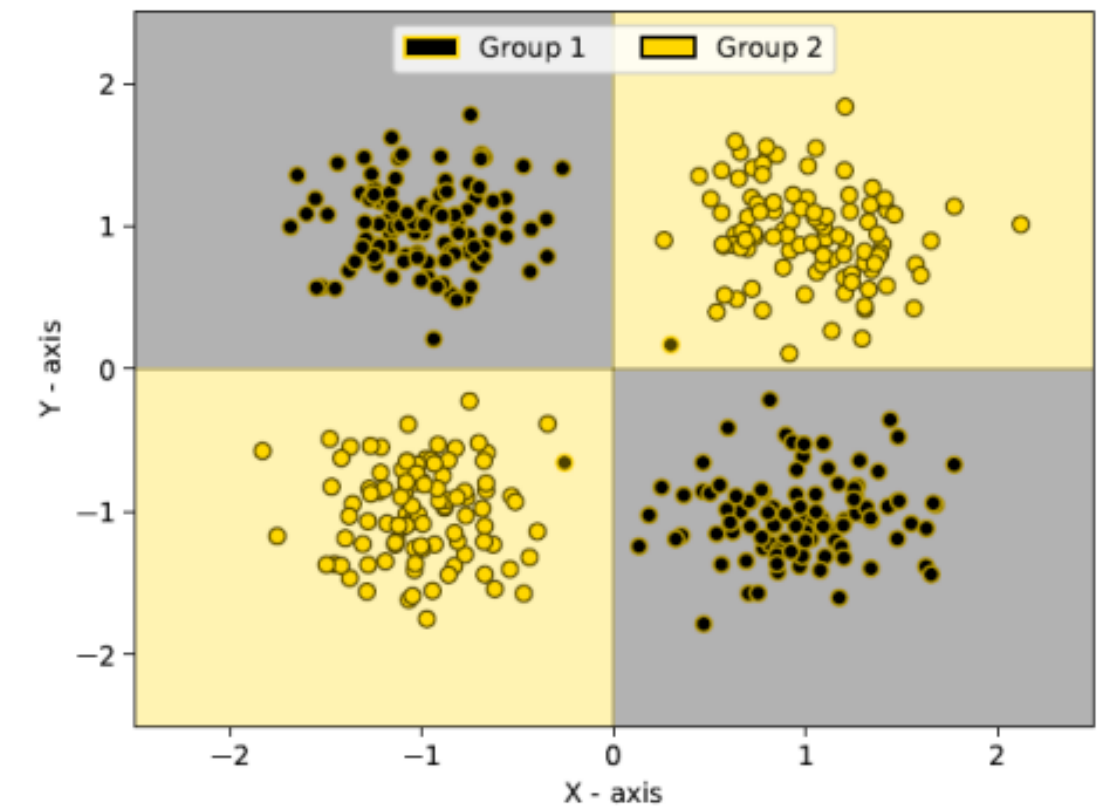
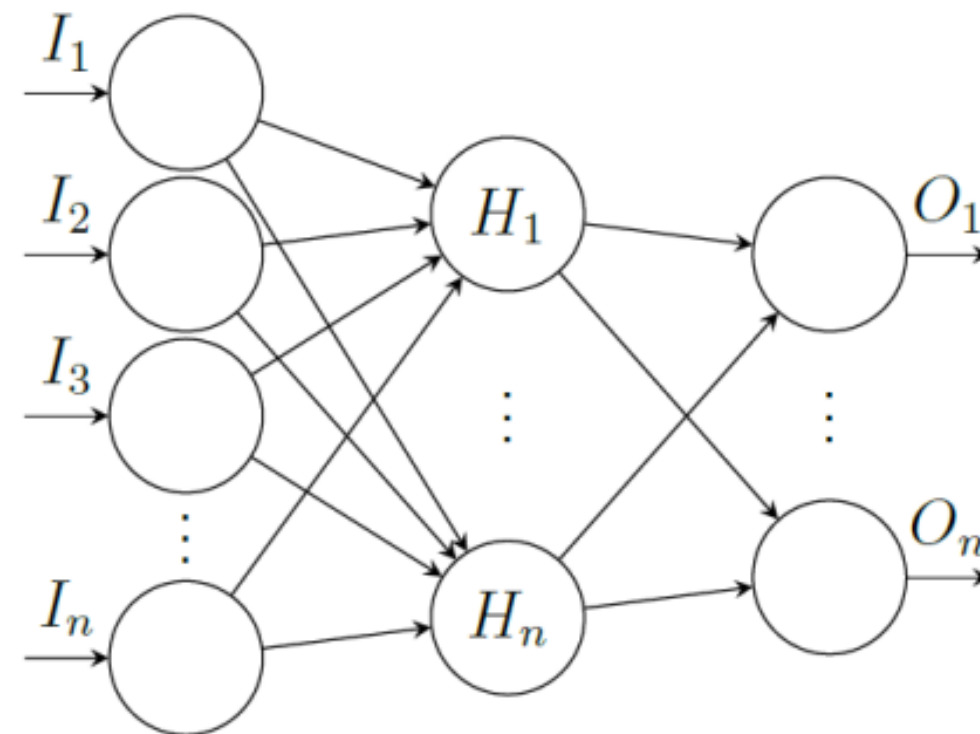
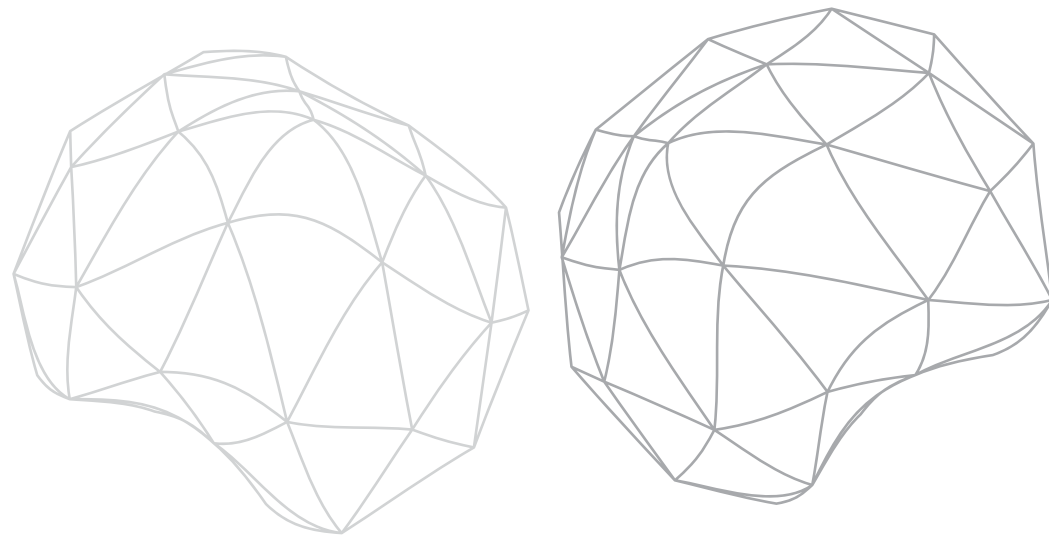
Input		Output
A	B	C
0	0	0
0	1	1
1	0	1
1	1	0



Classical Deep Learning



- A single perceptron can only solve linearly separable problems, requiring more sophisticated architectures for nonlinear ones, such as the multi layer perceptrons (MLP);
- The idea is to stack multiple layers of neurons, each using the same inputs, allowing the network to progressively transform the input data into more abstract representations with backpropagation, with each layer being computed as a matrix multiplication;
- The output is then fed into a non-linear function, the activation function, following a summation, via the **Wiener method**;



Classical Deep Learning

- The outputs from the final layer (logits) are often converted to probabilities via the **softmax function**

$$\sigma(z)_i = \frac{e^{\beta z_i}}{\sum_{j=1}^K e^{\beta z_j}}, \quad i = 1, \dots, K$$

where K is the number of logits, $\vec{z}$ is the vector of output logits and β a hyperparameter (temperature) that controls the smoothness of the function;

- Now the MLP is capable of classifying non linear data, as in the **XOR problem**, correctly separating the quadrants;
- MLPs must have at least **three layers**: the input, the output and a hidden one, having many matrix vector multiplications

$$f(x_n; W) = \sigma_{N_h} \left(W_{N_h} \sigma_{N_{h-1}} \left(\dots \sigma_1 (W_q X_n) \right) \right)$$

Classical Deep Learning

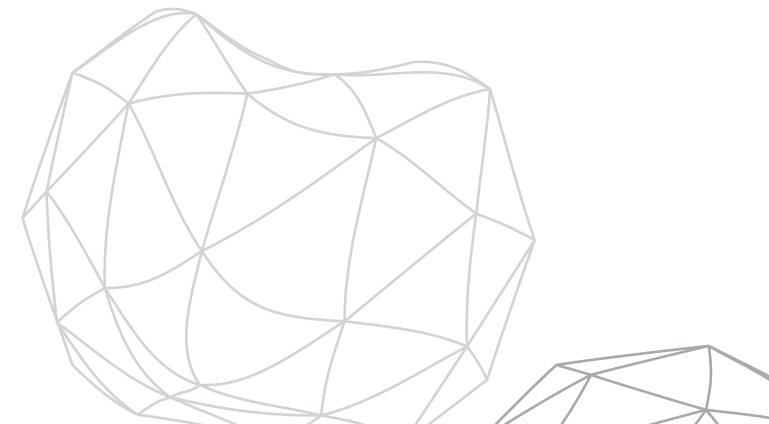
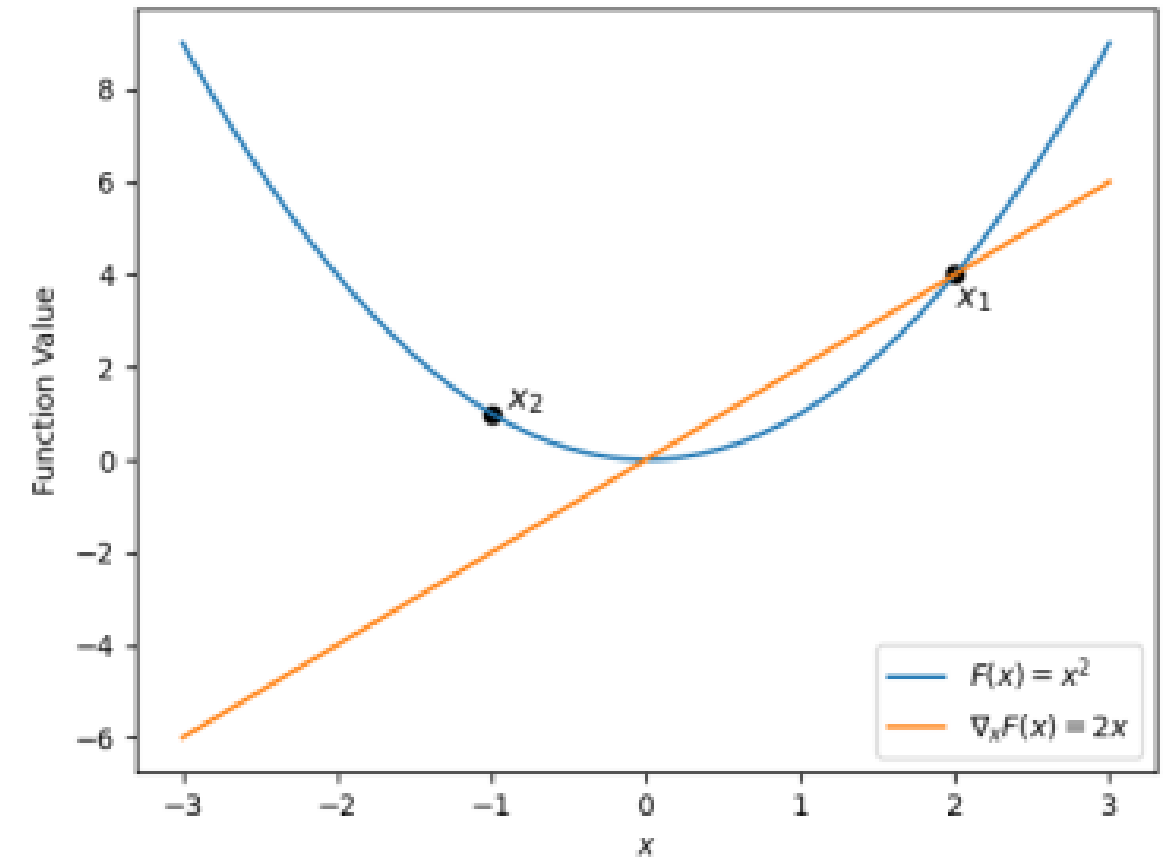
- **Gradient descent** is an optimization strategy widely used for fitting many different types of models and data, that try to find the minimum of a function $F(x)$, computing its gradient at x_1 and moving to the opposite direction of the actual minimum;
- Is repeated over many iterations via small steps in its direction, until a convergence point, via

$$x' = x - \lambda \nabla_x F(x)$$

- For machine learning it follows the same formula,

$$\mathcal{F}(W, X, Y) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(W, x_i, y_i)$$

where $\mathcal{L}(W, x_i, y_i)$ is the loss function, that depends on the model parameters;



Classical Deep Learning

- For perceptrons, as discussed earlier, the model with the mean squared error as the loss function yields

$$\mathcal{F}(w, X, y) = \frac{1}{2n} \sum_{i=1}^n (y_i - w^T x_i)^2$$

and the optimization is done solely on the model parameters w , generating the following update equation for the algorithm

$$w' = w - \frac{\lambda}{n} \left(\sum_{i=1}^n (y_i - w^T x_i) \right) x_i$$

and is commonly referred as the **batch gradient descent update**;

- There are other types of gradients used in ML, such as **stochastic gradient descent**, **mini-batch gradient descent**, etc.

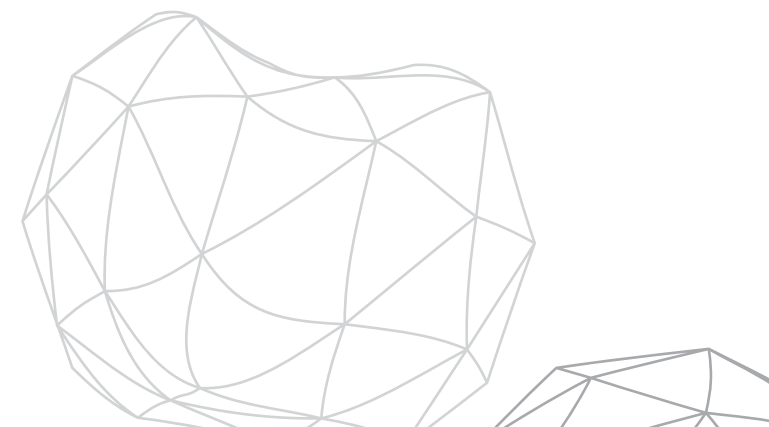
Classical Deep Learning

- For **neural networks** the sheer number of parameters and serial operations can make differentiating inefficient, so is common to use

$$f(x_n, y_n, W) = \frac{1}{2} \left(y_n - \sigma_{N_h} \left(W_{N_h} \sigma_{N_h-1} \left(\dots \sigma_1 (W_q x_n) \right) \right) \right)^2$$

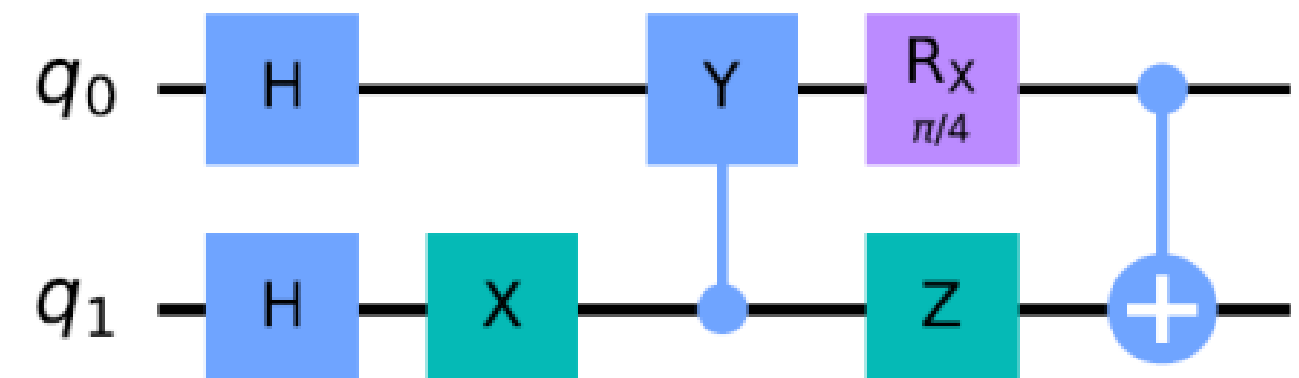
as the full loss function to be optimized;

- In **backpropagation** the **gradients are computed in a systematic fashion**, one layer at a time, as a large chain rule;
- The **per-layer loss** (local) gradient is computed backwards across layers of the network, with each parameter being updated in accordance.



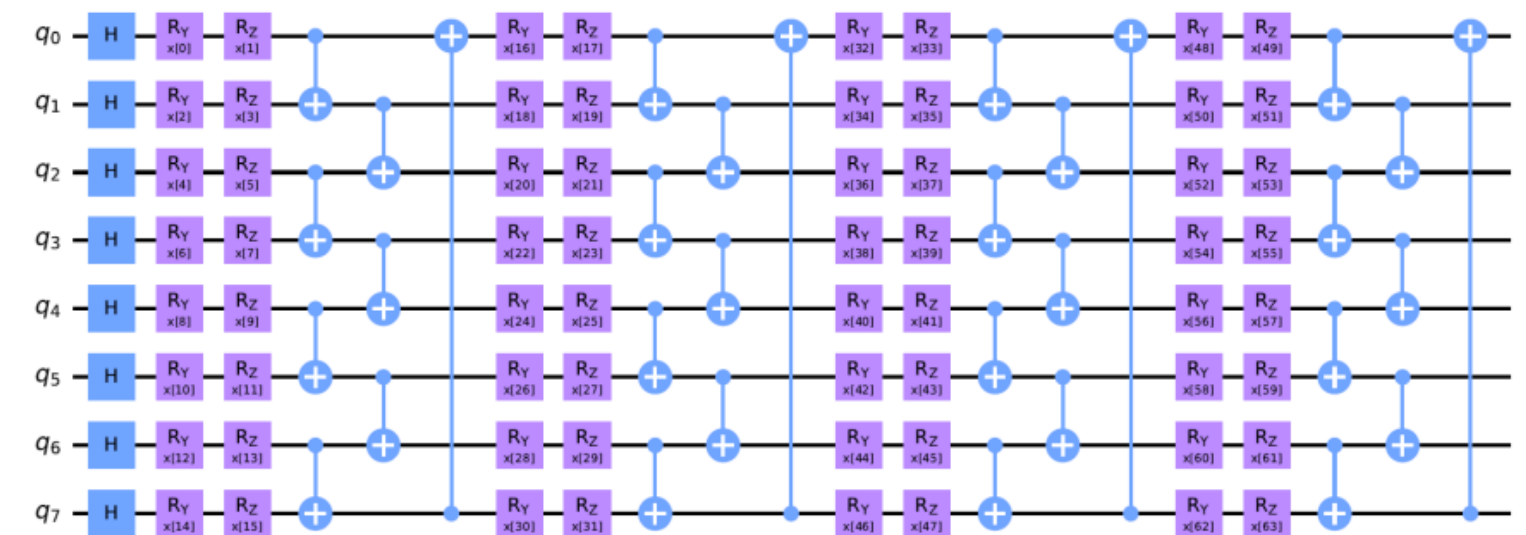
Quantum Machine Learning

- The currently dominant approach to quantum computing is to create a **quantum circuit**, similar to classical digital logic circuits in organization and structure, but with some differences;
- Instead of bits, initialized in a 0 state and passed through logic gates that may change their state to 1, we use qubits, similarly prepared in some initial state $|\psi\rangle$, which can exist in a superposition of states;
- A measurement in a quantum circuit collapses the wave function of the measured qubit, reducing it to a single classical output value;
- Since quantum gate operations must be unitary and reversible, the total number of qubits are conserved throughout the circuit;
- **One measurement is not enough** to deduce the state of a qubit, making it necessary to repeat the process to estimate its expectation value;



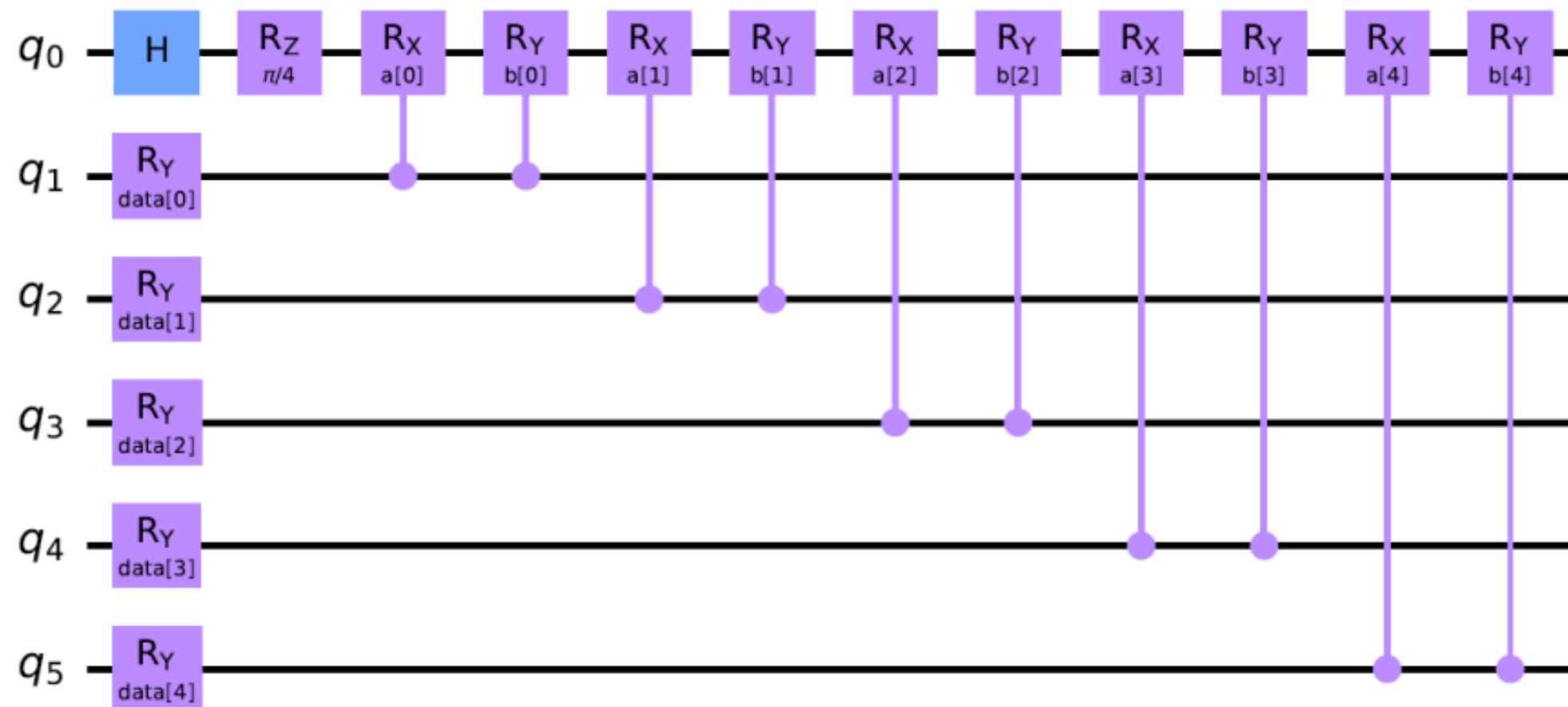
Quantum Machine Learning

- There are **two main strategies** to encode classical data to quantum computers;
- The **first** is to use **classical dimensionality reduction techniques** to reduce the dimension to match the number of qubits available, **converting the reduced dimension data to binary** values and vice-versa;
- **Another** technique used is gate encoding where the original floating-point data is **encoded directly into a quantum circuit with the use of rotation gates**, normalized in the range of $[0, \pi]$ or $[0, 2\pi)$;
- In the latter each dimension of the data is encoded by exactly **one rotation gate per qubit** during the encoding, which can be used by additional circuit elements for machine learning;
- For data with a high dimensionality is possible to use **block encoding**, containing many rotation gates applied to the same qubits;



Quantum Machine Learning

- Quantum machine learning is very similar to classical machine learning, since its a hybrid technology, with a few replacements where the error function to be optimized is partially calculated by a quantum computer;
- Data and readout qubits are utilized, with multi-qubit operations, like rotation gates, taking place to manipulate the readout qubits to some desired state corresponding to the input and in the end of each run the expectation value of the measured readout is used as the final output of the circuit and used to compute a loss function;



- One of the most used algorithms, similar to classical machine learning, is a variant of gradient descent, the parameter shift rule;

Quantum Machine Learning

- The **parameter-shift rule** allows the exact calculation of gradients in variational quantum circuits using shifted circuit evaluations;
- The **cost function** is an expectation value

$$C(\theta) = \langle \psi | \hat{U}_G^\dagger(\theta) \hat{A} \hat{U}_G(\theta) | \psi \rangle$$

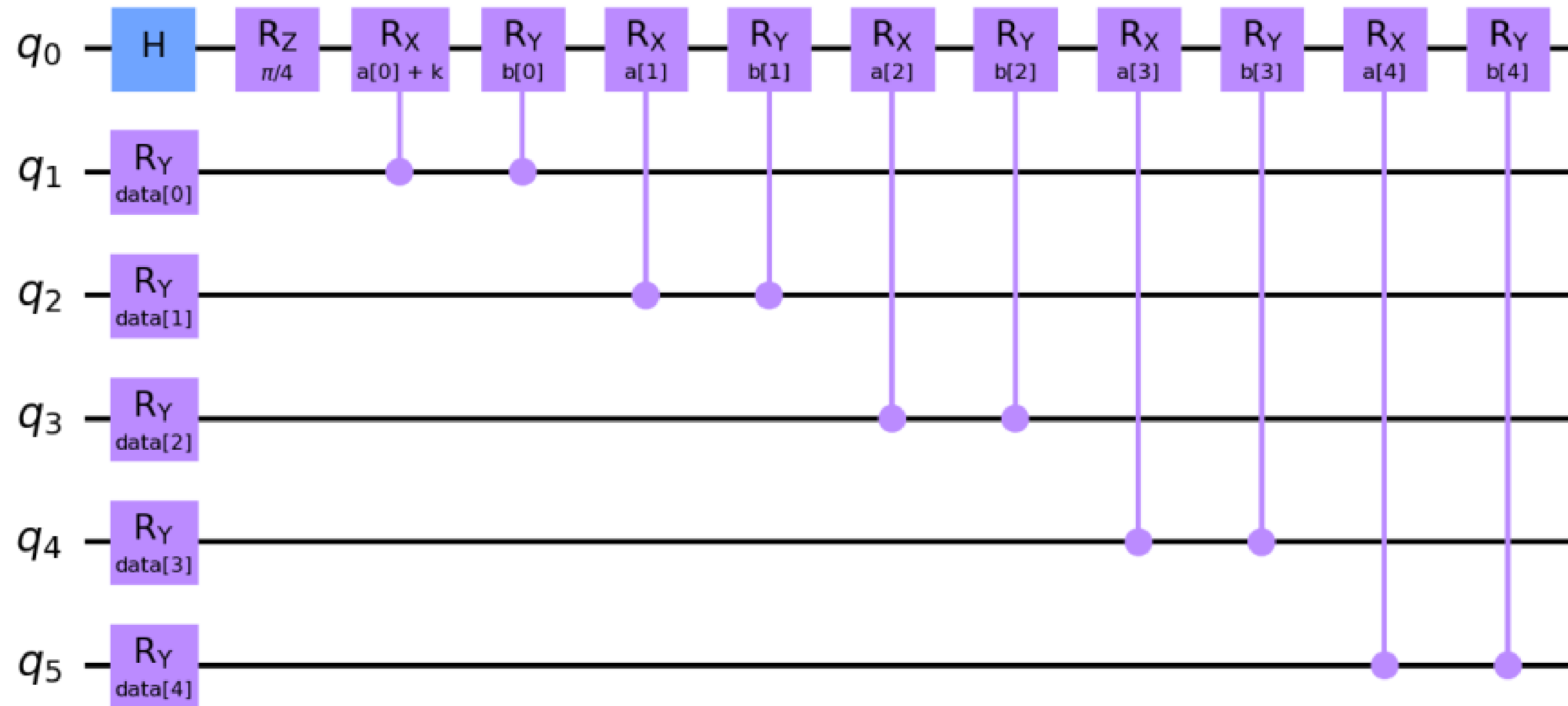
where $\hat{U}_G(\theta)$ is a parametrized quantum gate, $\hat{A}$ the measured observable and θ the trainable parameter;

- The gradient is obtained by differentiating this cost function and is **analytically obtained** as

$$\frac{dC(\theta)}{d\theta} = a \left[C\left(\theta + \frac{\pi}{2}\right) - C\left(\theta - \frac{\pi}{2}\right) \right]$$

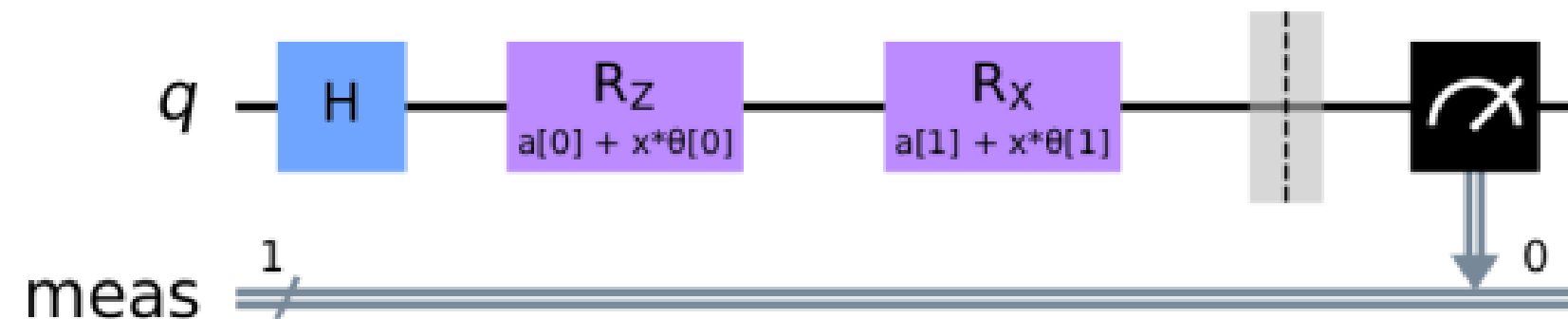
- The **same circuit is evaluated twice**, one with a positive parameter shift and the other with a negative shift and its difference gives the gradient to be optimized;

Quantum Machine Learning



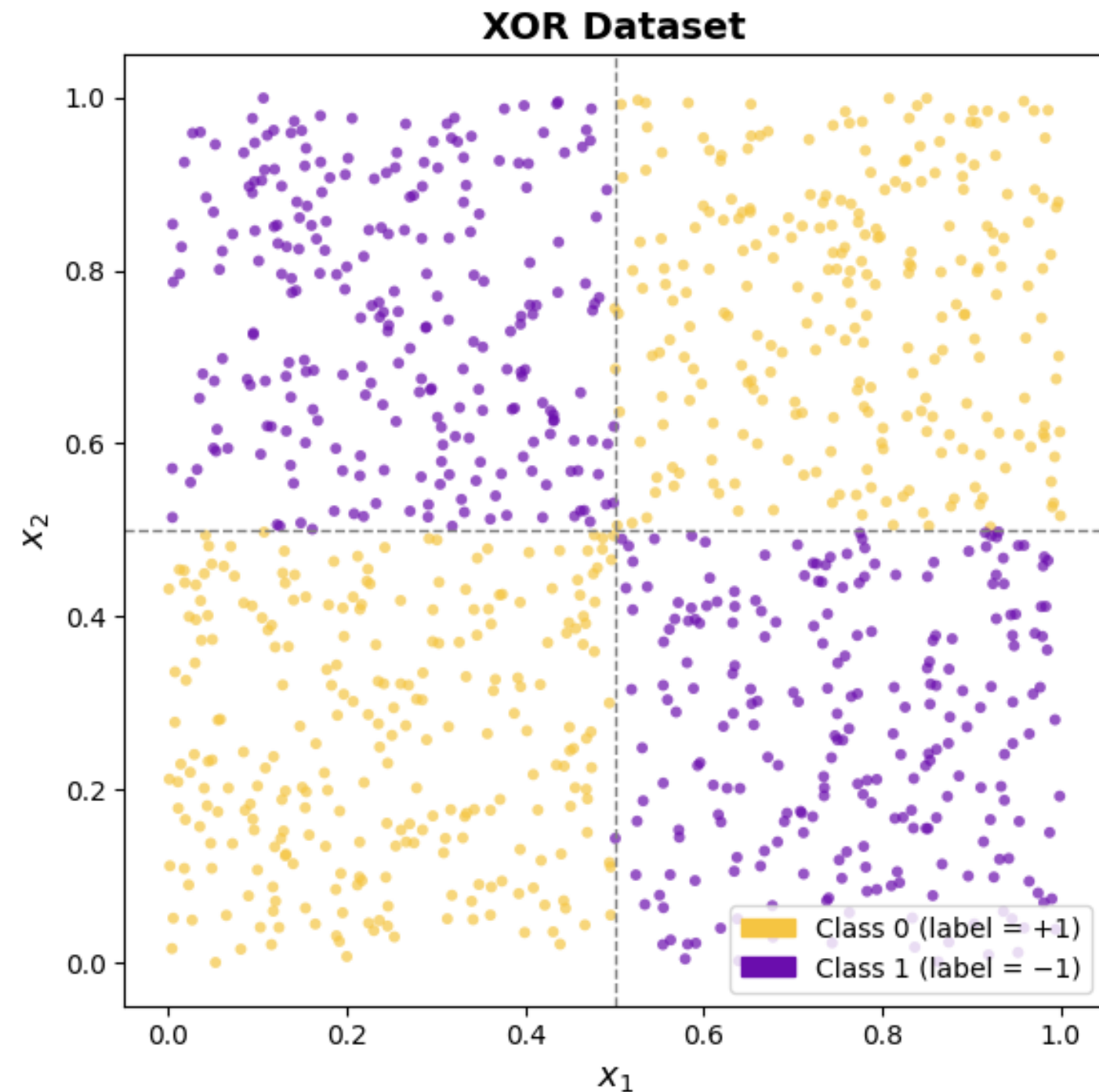
XOR Problem

- For the next slides we'll run **two tests to solve the XOR problem**, one using a **classical MLP model** and the other with a **quantum circuit** as the one presented by *de Grossu (2021)* with approximately the **same number of parameters**, to see the feasibility of quantum machine learning in general;
- The quantum circuit uses only **one qubit** and the parameter shift rule presented earlier, with **two rotations** θ_0 and θ_1 , as well as a **Hadamard gate** for parametrization;
- The qubit was **simulated in python using qiskit**, from IBM, and the models were run using **sklearn**;



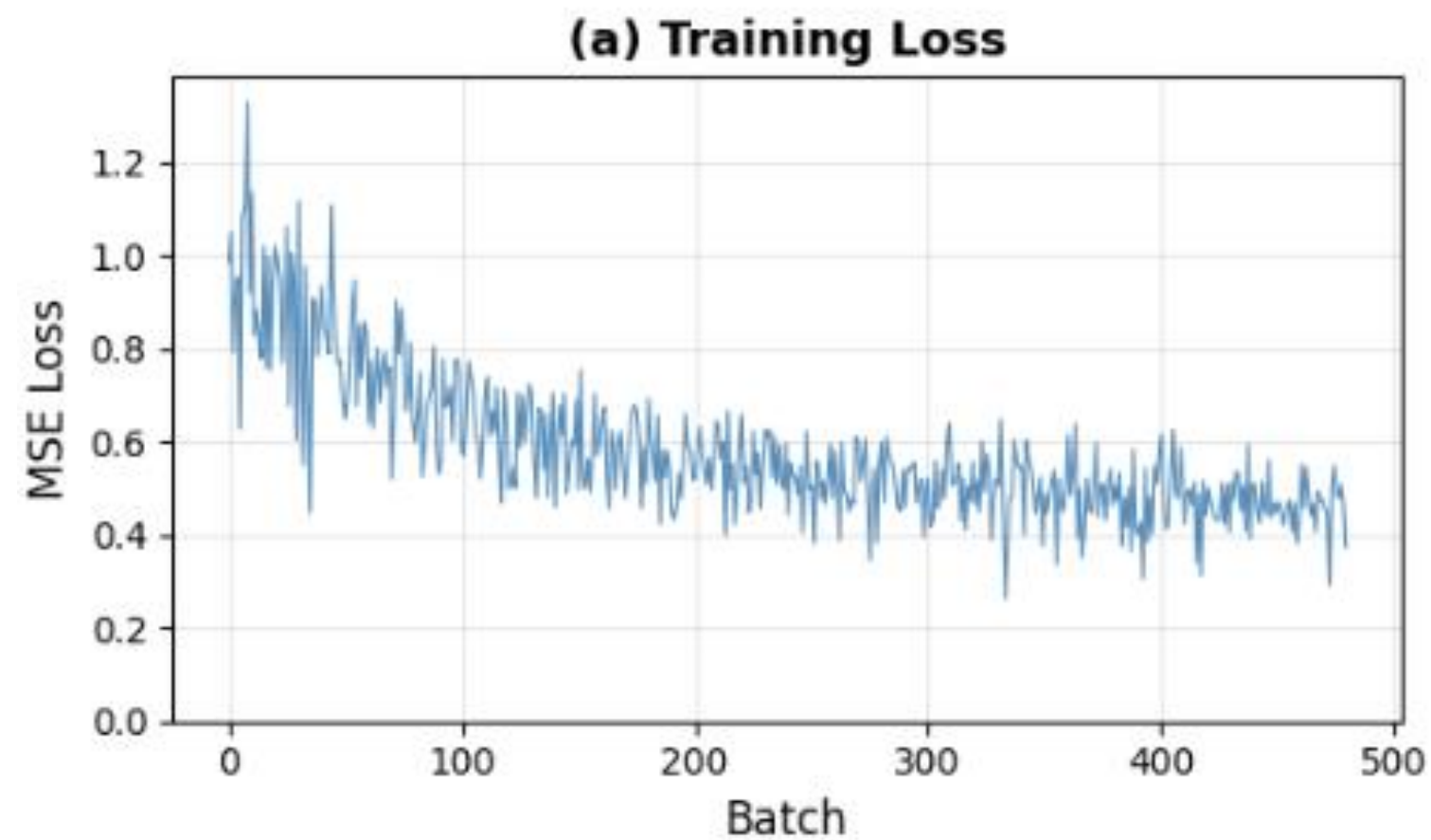
XOR Problem

- The **XOR dataset** was generated using numpy with a **random seed** and **1000 samples** as shown below;
- The dataset was split using **800 points for training**, **100 for validation** and **100 for testing** its accuracy;



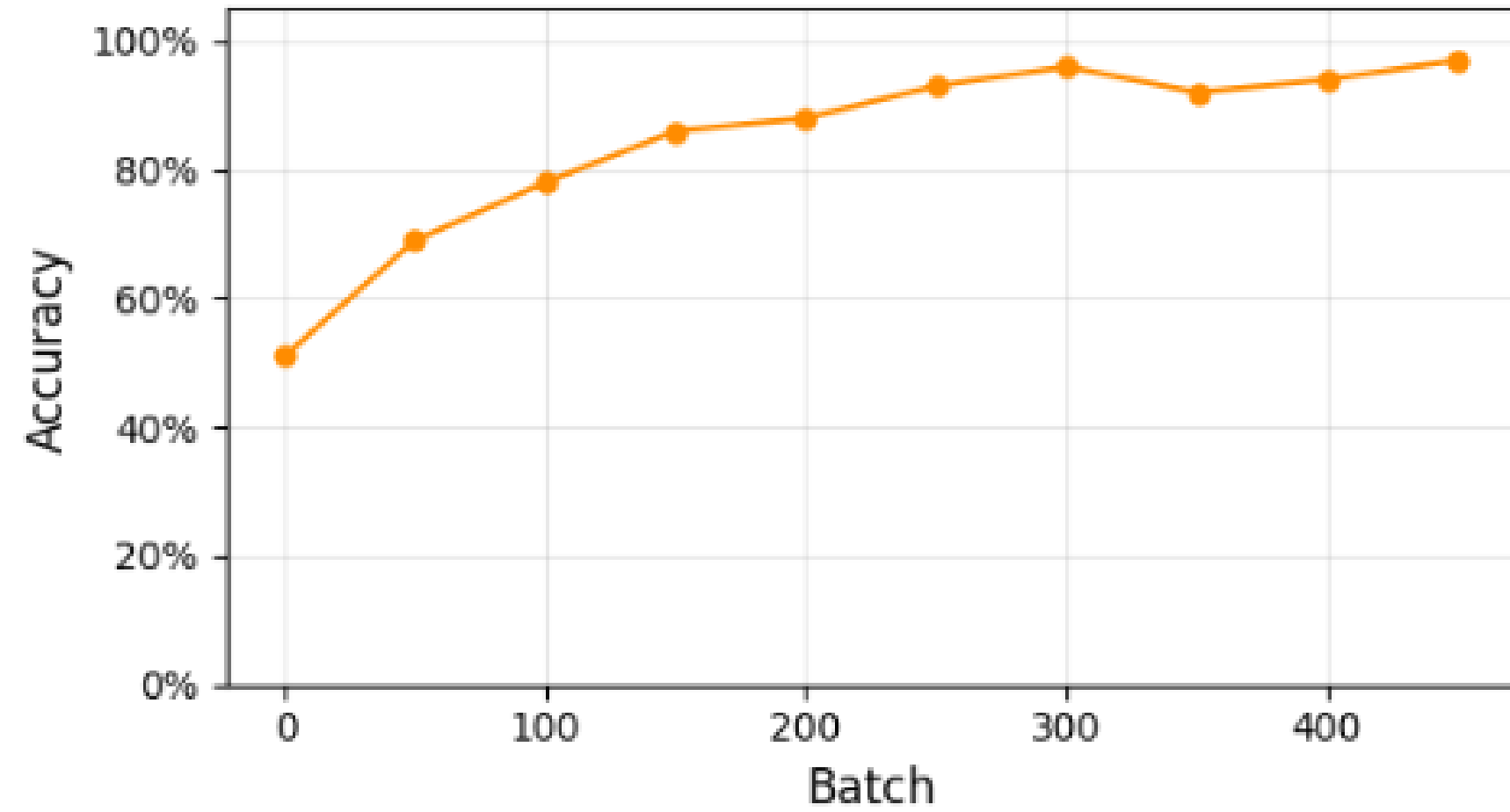
XOR Problem

- The quantum circuit was simulated on **1024 evaluations** using **AerSimulator** from qiskit;
- After many iterations to find the best training parameters, the circuit was trained using a **learning rate of 0.01, with a batch size of 25 and 15 epochs** (that represents the number of times the algorithm goes through the whole dataset), and with **random initial values**;
- After the training was completed, **the results obtained converged** to $\theta_0 = 2.44$, $\theta_1 = 2.00$, $\alpha_0 = 3.77$ and $\alpha_1 = 1.24$ with an **accuracy of 97%** in validation and **95%** in testing;

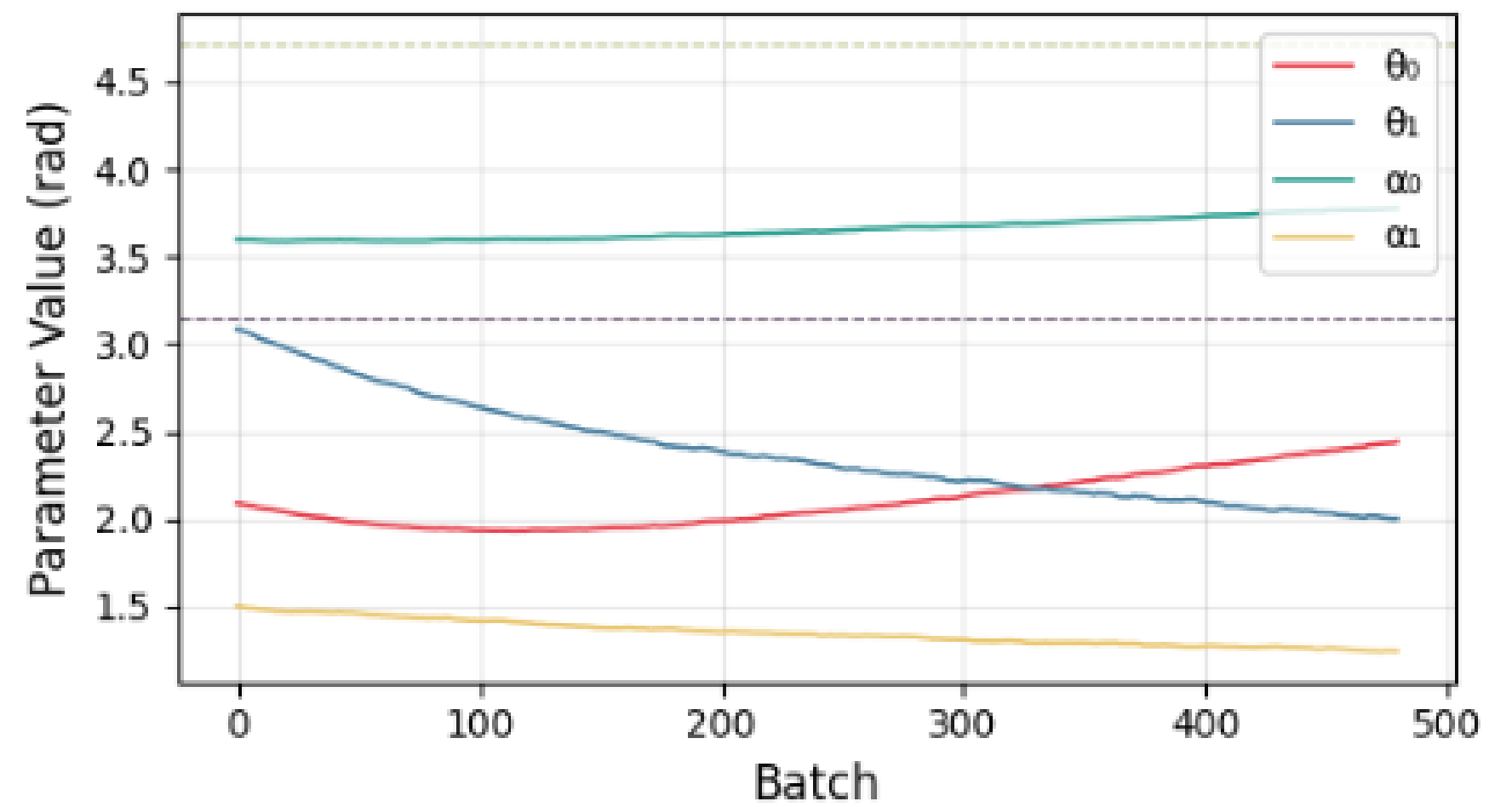


XOR Problem

(b) Validation Accuracy

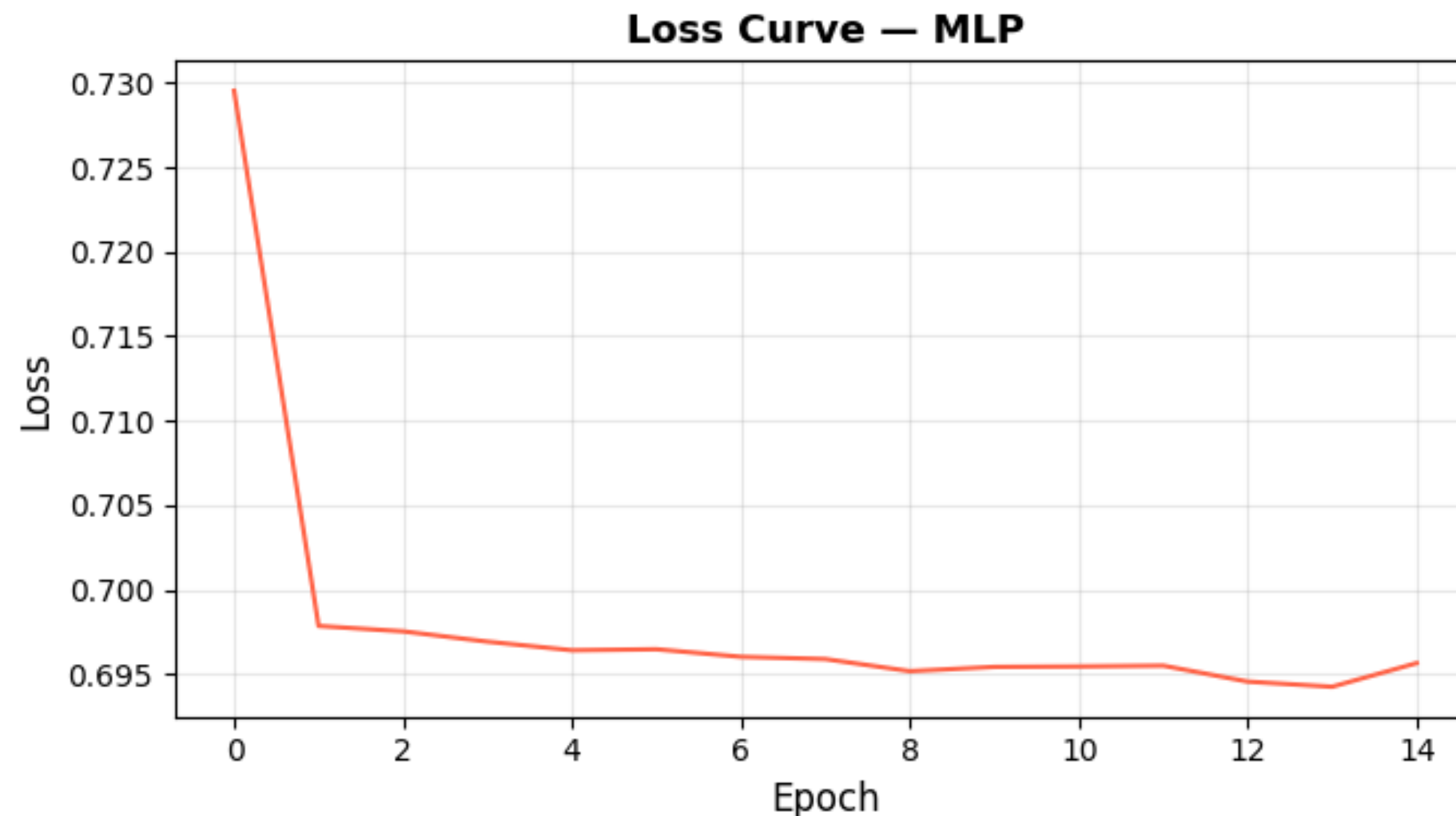


**(c) Parameters Convergence
(Dashed = Article ideal Values)**



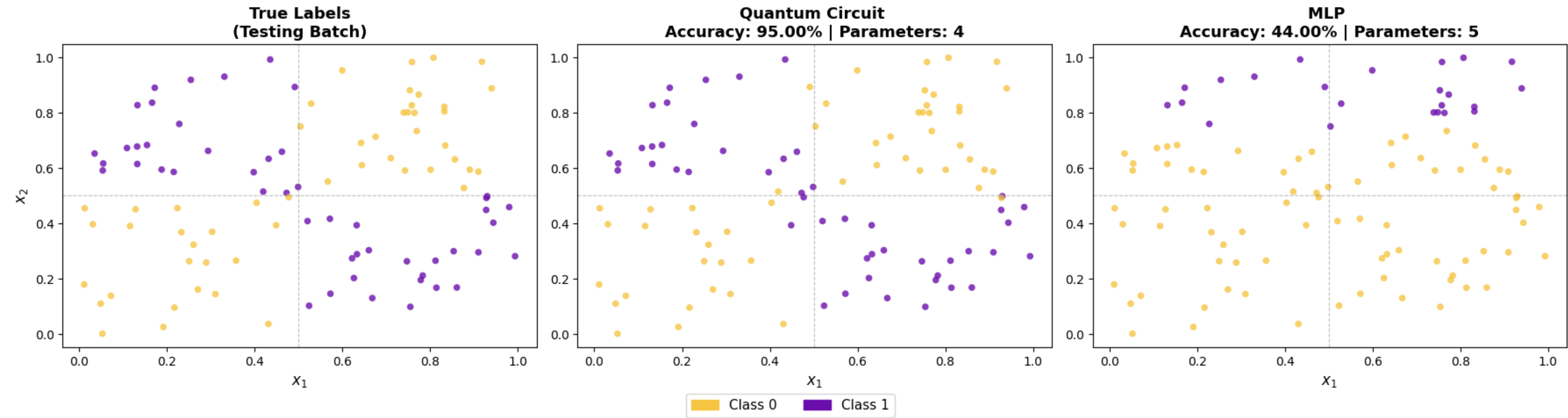
XOR Problem

- The **classical circuit** was run with one hidden layer of one neuron, to get the same approximate number of parameters, learning rate of 0.01, batch size of 25 and 15 iterations;
- The training was **much faster** but the **results were significantly worse**, obtaining a testing **accuracy** of **44%** due to the **low number of parameters**;

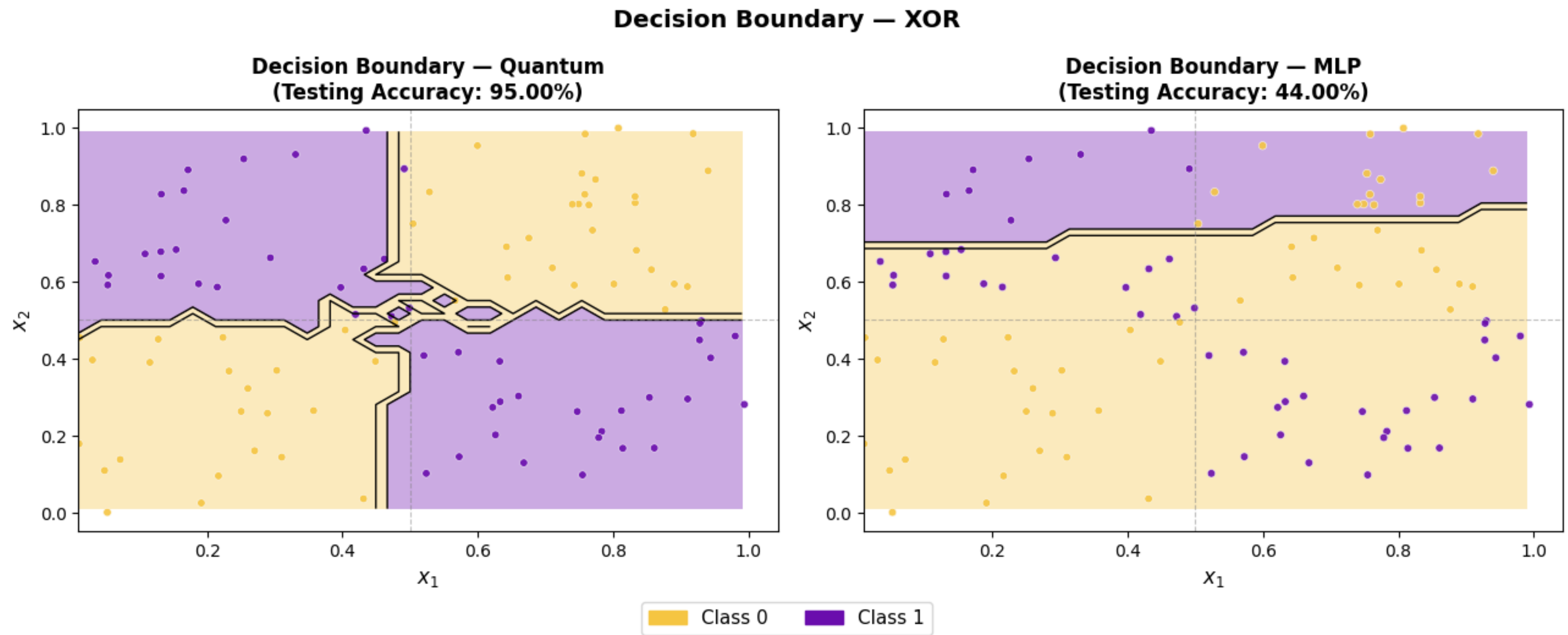


XOR Problem

XOR Problem - Quantum vs Classical



XOR Problem



Conclusion

- In this work, the following topics were presented:
- **Fundamental concepts of quantum mechanics**, including quantum phase, superposition, and entanglement;
- An **introduction to quantum gates** and their role in building the framework of quantum computation;
- The **basics of classical machine learning**, as well as the **fundamental concepts of quantum machine learning**;
- Finally, these concepts were applied to an example problem that illustrates the **potential advantages of quantum machine learning over classical approaches** in certain tasks, although studies in this area, such as the one presented in this work, are still at an **early stage of development**;
- With the increasing capabilities of quantum computers, **quantum machine learning shows great potential for solving complex problems** across several domains, which justifies **further research into the quantum advantage** provided by quantum machine learning.

References

- BYRNE, Dominic; COOK, Matthew G.; EVANS, Ethan N. **A Quick Introduction to Quantum Machine Learning for Non-Practitioners**. arXiv:2402.14694, 22 fev. 2024. Disponível em: <https://arxiv.org/abs/2402.14694>. Acesso em: 18 mai. 2026.
- GROSSU, Ioan Valeriu. **Single qubit neural quantum circuit for solving exclusive-OR**. MethodsX, v. 8, p. 101573, 6 nov. 2021. DOI: 10.1016/j.mex.2021.101573.
- NIELSEN, Michael A.; CHUANG, Isaac L. **Quantum Computation and Quantum Information**. 10. ed. Cambridge: Cambridge University Press, 2010. 702 p. ISBN 978-1-107-00217-3.
- COHEN-TANNOUDJI, Claude; DIU, Bernard; LALOË, Franck. **Quantum Mechanics**. 2. ed. Weinheim: Wiley-VCH, 2019. 2 v. ISBN 978-3-527-34553-3.
- ROSENBLATT, Frank. **The Perceptron: A Perceiving and Recognizing Automaton**. Buffalo: Cornell Aeronautical Laboratory, 1957. Relatório Técnico n. 85-460-1.
- HORNIK, Kurt; STINCHCOMBE, Maxwell; WHITE, Halbert. **Multilayer feedforward networks are universal approximators**. Neural Networks, v. 2, n. 5, p. 359–366, 1989. DOI: 10.1016/0893-6080(89)90020-8.
- BERGHOLM, Ville et al. **PennyLane: Automatic differentiation of hybrid quantum-classical computations**. arXiv:1811.04968, 2018. Disponível em: <https://arxiv.org/abs/1811.04968>. Acesso em: 18 mai. 2026.
- SCHULD, Maria et al. **Evaluating analytic gradients on quantum hardware**. Physical Review A, v. 99, n. 3, p. 032331, 21 mar. 2019. DOI: 10.1103/PhysRevA.99.032331.

